

SwiftShip Tracker: Project Report

Salesforce Developer project | Team lead: Praveen P | Submission deadline: Saturday, 3 October 2026

GitHub: <https://github.com/praveen0-100/-swiftship-tracker>

1. Introduction

Overview. SwiftShip Tracker is a Salesforce CRM solution for parcel booking, shipment tracking and delivery management. It covers the full lifecycle: booking, dispatch, in-transit tracking, out for delivery, and delivery confirmation. Customers see shipment information; delivery agents update parcel status.

Purpose.

- Centralize parcel and delivery information
- Simplify booking and tracking
- Give real-time shipment visibility
- Automate delivery updates and notifications
- Enable conversational tracking with Agentforce and Prompt Builder
- Provide role-based access for customers, agents, support staff and admins

2. Ideation Phase

2.1 Problem statements

PS	I am	I'm trying to	But	Because	Which makes me feel
PS-1	Customer / Sender	book a parcel easily	booking requires support or multiple apps	parcel operations are not centralized	frustrated
PS-2	Customer / Sender	track my parcel	there is no simple real-time tracking	shipment information is hard to access	confused
PS-3	Delivery Agent	update parcel status	updates are manual	there is no centralized operational tracking	concerned
PS-4	Customer	receive delivery updates	notifications are not sent at every stage	communication is not automated	uncertain
PS-5	Admin	monitor parcel operations	flow and performance are hard to see	data is not centrally reported	concerned

2.2 Empathy map (Customer / Sender)

- **Think & feel:** wants to book and track without repeatedly contacting support
- **Hear:** delivery information and status updates from the parcel service
- **See:** parcel details, tracking status, estimated delivery date
- **Say & do:** books parcels, checks tracking, asks for updates
- **Pain:** delays, confusion, no easy tracking, dependence on support
- **Gain:** real-time visibility, self-service tracking, timely notifications

2.3 Brainstorming

Ideas considered: manual registers, spreadsheet tracking, a standalone parcel portal, Salesforce CRM with Agentforce. **Selected: Salesforce CRM with Agentforce** for centralized management, Flow automation, real-time visibility, conversational AI, role-based security, and reports.

3. Requirement Analysis

3.1 Customer journey

Parcel Booking > Parcel Record Creation > Dispatch & Shipment > In-Transit Tracking > Out for Delivery > Delivery Confirmation > Customer Notification > Feedback / Support

3.2 Requirements

FR	Requirement	Implemented by
FR-1	Parcel booking and record creation	Parcel__c , SwiftShip Tracker app
FR-2	Parcel status tracking	Parcel__c.Status__c
FR-3	Weight management	Parcel__c.Weight__c , trigger validation
FR-4	Estimated delivery date	Estimated_Delivery_Date__c , trigger validation
FR-5	Sender information	Sender__c
FR-6	Receiver information	Receiver__c
FR-7	Delivery and location tracking	Delivery__c.Current_Location__c
FR-8	Agent status updates	Swift_Ship permission set, Parcel record edit
FR-9	Automated notifications	ParcelTriggerHandler status-change emails, OverdueParcelBatch
FR-10	AI-powered tracking	Agentforce + Parcel_Details flow
FR-11	Reports and dashboards	see docs/manual-steps.md
FR-12	Customer access	Experience Cloud (future scope in this prototype)

NFR	Description
Usability	Intuitive Lightning app and tabs
Security	Permission sets, field-level access
Reliability	Validation keeps status and dates accurate
Performance	Bulkified trigger and batch (200 per scope)
Availability	Cloud-based Salesforce
Scalability	Batch Apex, lookup relationships
Data integrity	Required fields, relationship-based data
Maintainability	Handler pattern, configurable flow

3.3 Data flow

Customer/Sender > Parcel Booking > Parcel Record > Sender + Receiver > Delivery Record > Status Update > Location & ETA > Notifications > Tracking / Agentforce > Reports & Dashboards

AI flow: Customer query > Agentforce > Parcel_Details flow / Prompt Builder > Parcel record > Status, weight, ETA > conversational response

3.4 Technology stack

Layer	Technology
Platform	Salesforce Developer Edition
UI	Lightning App, Lightning record pages
Database	Custom objects
Logic	Flow, Apex trigger, Batch Apex
AI	Agentforce, Prompt Builder
Notifications	Apex email
Security	Permission sets, field-level security
Reporting	Reports and dashboards

4. Project Design

4.1 Problem-solution fit

Problem	Solution	Benefit
Booking is difficult	Parcel management in Salesforce	Centralized booking
Tracking needs support	Agentforce + Prompt Builder	Self-service tracking
Manual status updates	Flow and Apex	Automated processes
Communication gaps	Email on every status change	Timely notifications
Scattered data	Parcel, Delivery, Sender, Receiver objects	Structured data
Hard to monitor	Reports and dashboards	Better visibility
Different access needs	Permission sets	Secure access

4.2 Proposed solution

Problem: inefficient parcel booking and tracking. Solution: Salesforce parcel management system. Innovation: conversational tracking through Agentforce with automated Flow and Apex. Impact: centralized booking, real-time visibility, timely notifications. Scalability: supports growth and external logistics integration.

4.3 Architecture

Users (Customer, Delivery Agent, Support, Admin) > Salesforce Lightning > SwiftShip Tracker platform: **Data** (Parcel, Delivery, Sender, Receiver) | **Automation** (Flow, Apex, Batch Apex, email) | **AI** (Agentforce, Prompt Builder, Parcel Tracking Agent) | **Security** (permission sets, FLS) > Reports & Dashboards

5. Project Planning

Agile, sprint-based, epics > stories > story points.

Sprint	Epic	Story	Points	Priority	Member
1	Developer setup	USN-1 Create and configure the Salesforce environment	3	High	Praveen P
2	Data modeling	USN-2 Create Parcel, Delivery, Sender, Receiver objects and relationships	5	High	Priyan K

2	Data modeling	USN-3 Tabs and Lightning app	5	High	Nishanth S
3	Automation	USN-4 Validate parcel details and mandatory fields	3	High	Nandhakumar A
3	Automation	USN-5 Flows to update parcel status and delivery info	3	High	Nandhakumar A
4	Apex	USN-6 Triggers and classes for parcel actions	5	High	Praveen P
4	Apex	USN-7 Batch Apex for overdue parcels	5	High	Priyan K
5	Reports	USN-8 Status and delivery performance reports	4	High	Nishanth S
6	Dashboards & AI	USN-9 Dashboards and Agentforce tracking	4	Medium	Nandhakumar A

Project tracker, velocity and burndown

Sprint dates follow the actual build log (all work done 30 Sep to 1 Oct 2026, ahead of the 3 Oct deadline).

Sprint	Total story points	Duration	Start	End (planned)	Points completed	Release date (actual)
Sprint-1	3	1 day	30 Sep 2026	30 Sep 2026	3	30 Sep 2026
Sprint-2	10	1 day	30 Sep 2026	30 Sep 2026	10	30 Sep 2026
Sprint-3	6	1 day	30 Sep 2026	30 Sep 2026	6	30 Sep 2026
Sprint-4	10	1 day	30 Sep 2026	30 Sep 2026	10	30 Sep 2026
Sprint-5	4	1 day	1 Oct 2026	1 Oct 2026	4	1 Oct 2026
Sprint-6	4	1 day	1 Oct 2026	1 Oct 2026	4	1 Oct 2026

Total: 37 story points over 6 sprints, average velocity about 6.2 points per sprint.

Team

Name	Role
Praveen P	Team Lead (Member1 in the backlog above)
Priyan K	Team Member (Member2)
Nishanth S	Team Member (Member3)
Nandhakumar A	Team Member (Member4)
Prakash V	Team Member

6. Development Phases

Phase 1: Requirement analysis. Personas (Customer, Agent, Support, Admin), data model, use cases, tool selection.

Phase 2: Backend. Objects and fields (`force-app/main/default/objects`), tabs, app, `Parcel_Details` flow.

Phase 3: UI and Agentforce. `Swift_Ship` permission set, Agentforce default agent, "Parcel Updates" subagent, flow action, agent renamed **SwiftShip Tracker**.

Phase 4: Testing. Verify the agent lists, subagent and flow are active; preview "Track parcel P-001"; confirm data on the Parcel record. Apex tests: 5/5 pass (`SwiftShipTests`).

Phase 5: Deployment and maintenance. Validated and deployed to a Developer Org with `sf project deploy start`. Sandbox/UAT and CI/CD are out of scope for this prototype. Maintenance: monitor flow failures, refine prompts and subagent topics, keep parcel data current.

7. Functional and Performance Testing

Evidence is in `docs/test-evidence.md` (5/5 Apex tests passing, validation errors for weight 0 and past dates, flow output for P-001, org records) and `docs/screenshots/`:

- `01-parcel-object-fields.jpg`: Parcel object fields
- `02-parcel-record-P-001.jpg`: Parcel record with auto-number ID
- `03-parcel-details.png`: Parcel Details flow in Flow Builder (Active)
- `04-retrieve-parcel-details.png`: Retrieve Parcel Details prompt template
- `10-retrieve-parcel-details-book-box.png`: Prompt Builder preview of Version 1 (resolved prompt filled with P-001 data). Version 1 was later replaced by the active Version 2, whose correct output is in `docs/test-evidence.md` section 7.
- `04-report-parcels-by-status.jpg` and `05-report-overdue-parcels.jpg`: the two reports
- `06-validation-error-weight-zero.jpg`: red validation error when saving Weight = 0
- `07-dashboard-swiftship-operations.jpg`: SwiftShip Operations dashboard
- `08-agent-preview-track-P-001.jpg`: agent answering "Track parcel P-001" (subagent, action and grounded output in the trace)
- `09-agent-active.jpg`: agent SwiftShip Tracker, Version 1 (Active)

The `Retrieve Parcel Details` prompt template is Published in the org. The agent definition is stored in the repo under `force-app/main/default/aiAuthoringBundles/SwiftShip_Tracker`.

8. Advantages and Disadvantages

Advantages: centralized data, real-time visibility, automated processes, conversational tracking, less manual effort, timely notifications, scalable.

Disadvantages: needs accurate data, depends on configuration, Agentforce quality depends on prompts, integrations add complexity, flows need upkeep, users need access.

9. Conclusion

SwiftShip Tracker shows how Salesforce objects, Flow, Apex and Agentforce combine into a centralized parcel management and tracking solution, letting customers get shipment information without manual support.

10. Future Scope

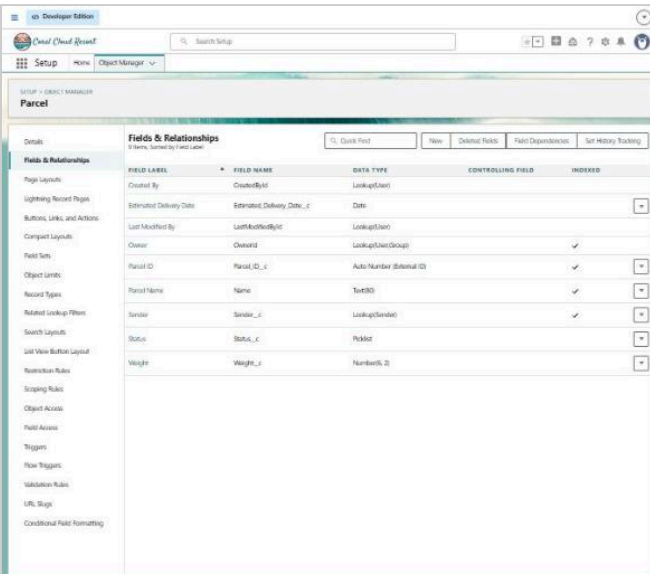
Broader Agentforce queries, richer real-time tracking, Experience Cloud self-service, delivery analytics, external logistics integration, better prompts, sandbox and CI/CD.

11. Appendix

A. Custom objects: Parcel, Delivery, Sender, Receiver | B. Automation: Flow, Apex trigger, email | C. Apex: `ParcelTriggerHandler`, `OverdueParcelBatch`, `SwiftShipTests` | D. AI: Agentforce, Prompt Builder | E. UI: Lightning app, tabs | F. Security: permission set | G. Analytics: reports and dashboards

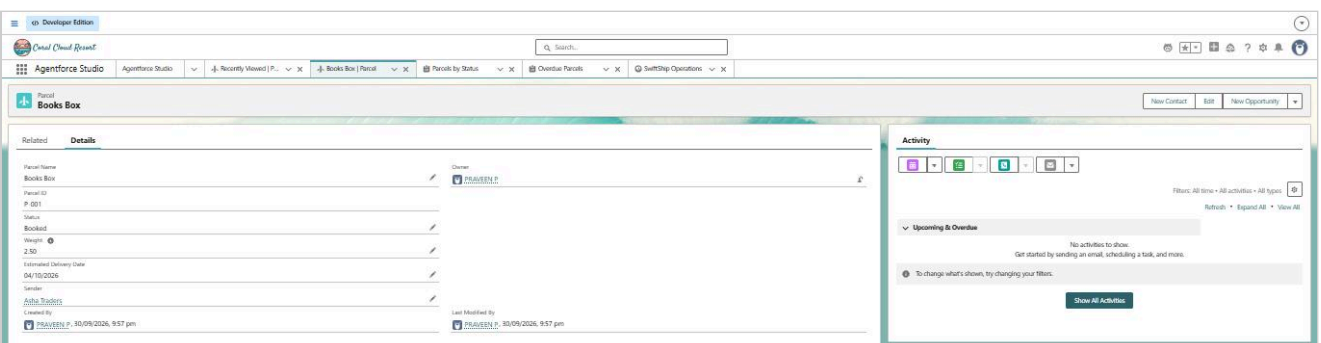
12. Screenshots

Figure 1: Parcel object: fields and relationships



FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Estimated Delivery Date	Estimated_Delivery_Date__c	Date		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User/Group)		✓
Parcel ID	Parcel_ID__c	Auto Number (18-Digit)		✓
Parcel Name	Name	Text(80)		✓
Service	Service__c	Lookup(Service)		✓
Status	Status__c	Picklist		
Weight	Weight__c	Number(2)		

Figure 2: Parcel record P-001 with all fields



Parcel Details

Parcel Name: Books Box

Parcel ID: P-001

Status: Booked

Weight: 2.50

Estimated Delivery Date: 04/10/2026

Service: Active Trainers

Created By: PRAVEEN P. 30/09/2026, 9:57 pm

Activity

Filters: All types • All activities • All types

Upcoming & Overdue

No activities to show.

Get started by sending an email, scheduling a task, and more.

To change what's shown, try changing your filters.

Show All Activities

Figure 3: Parcel Details flow in Flow Builder (Active)

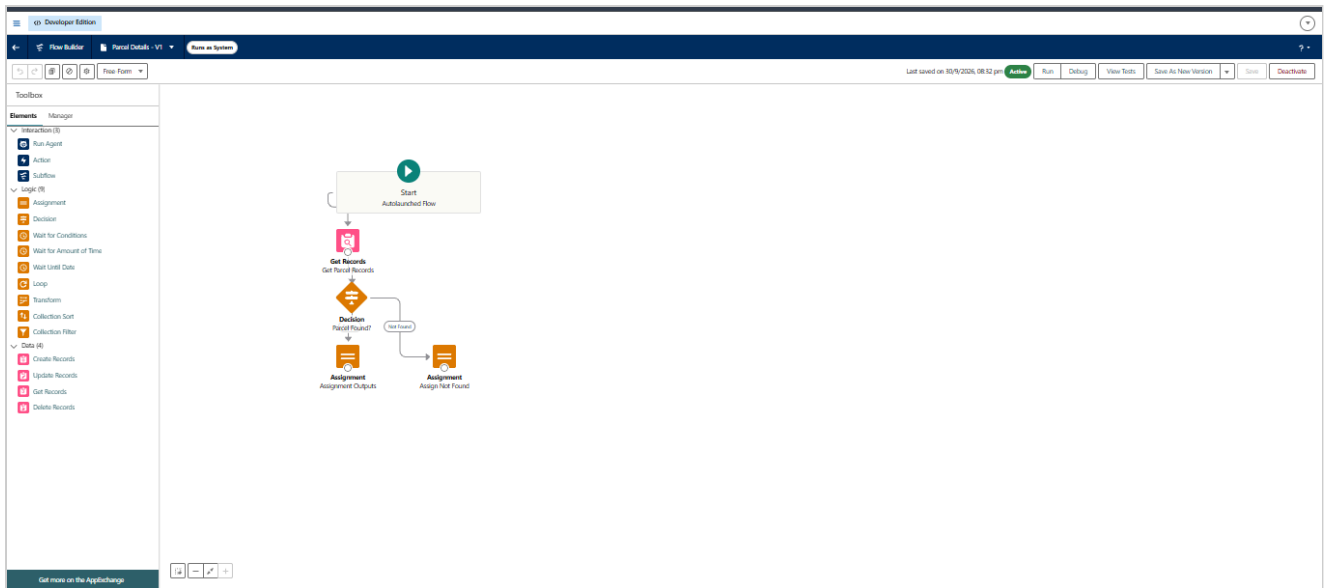


Figure 4: Retrieve Parcel Details prompt template

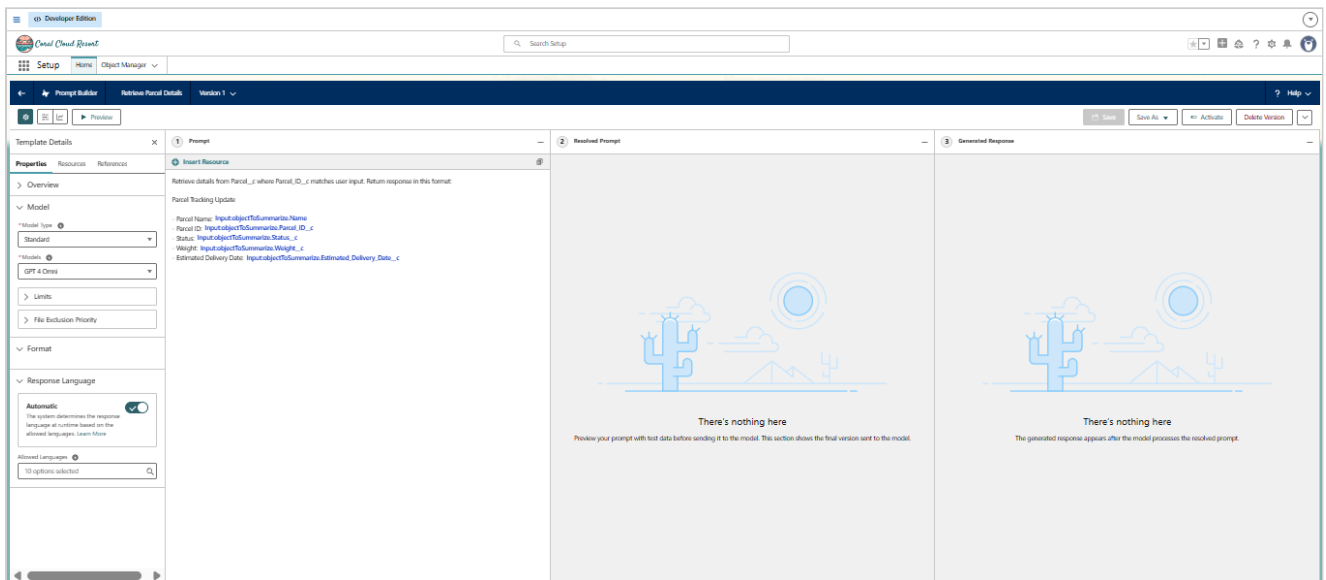


Figure 5: Report: Parcels by Status

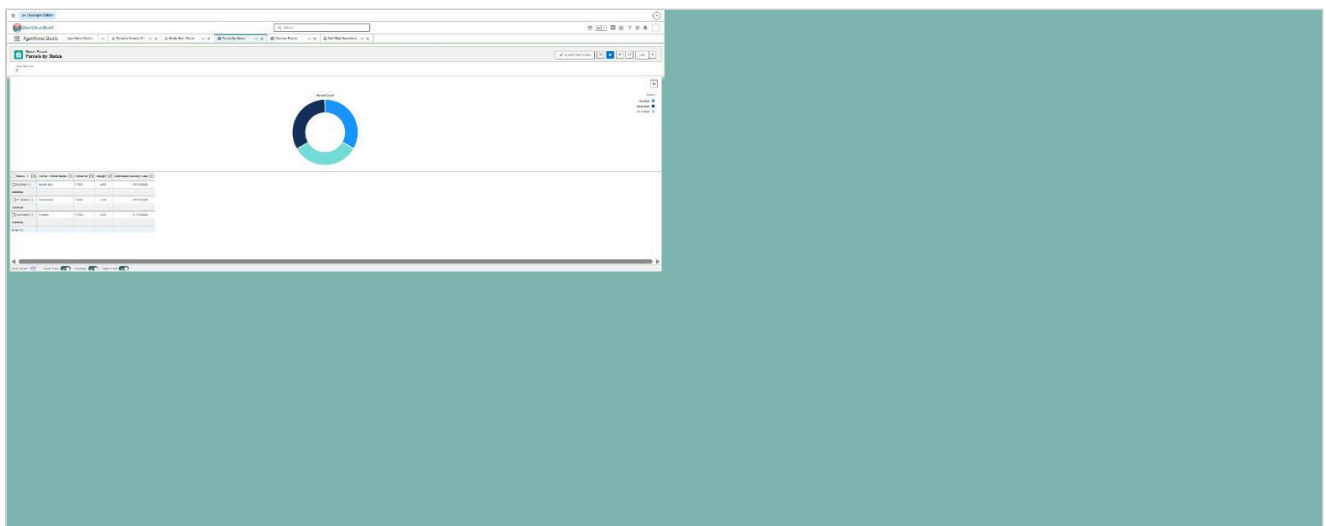


Figure 6: Report: Overdue Parcels

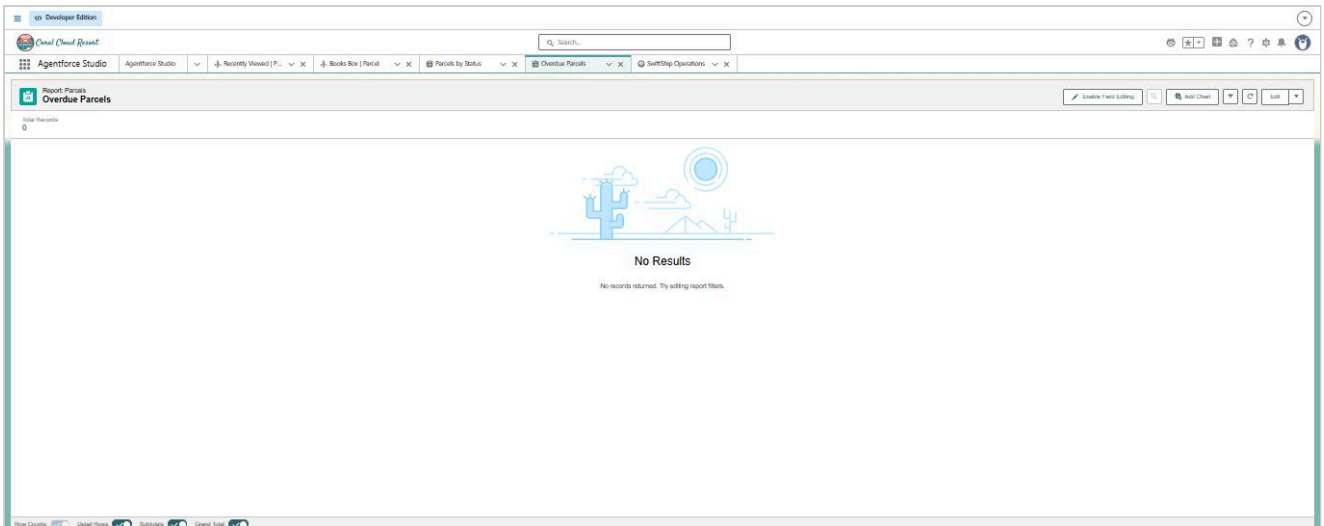


Figure 7: Validation error when saving Weight = 0

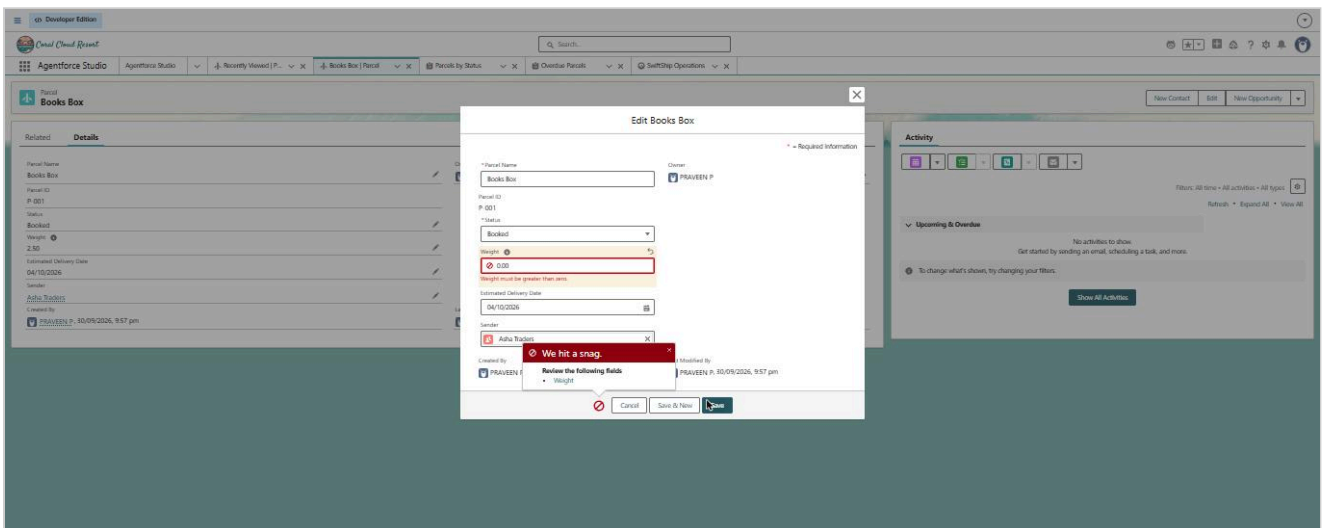


Figure 8: Dashboard: SwiftShip Operations

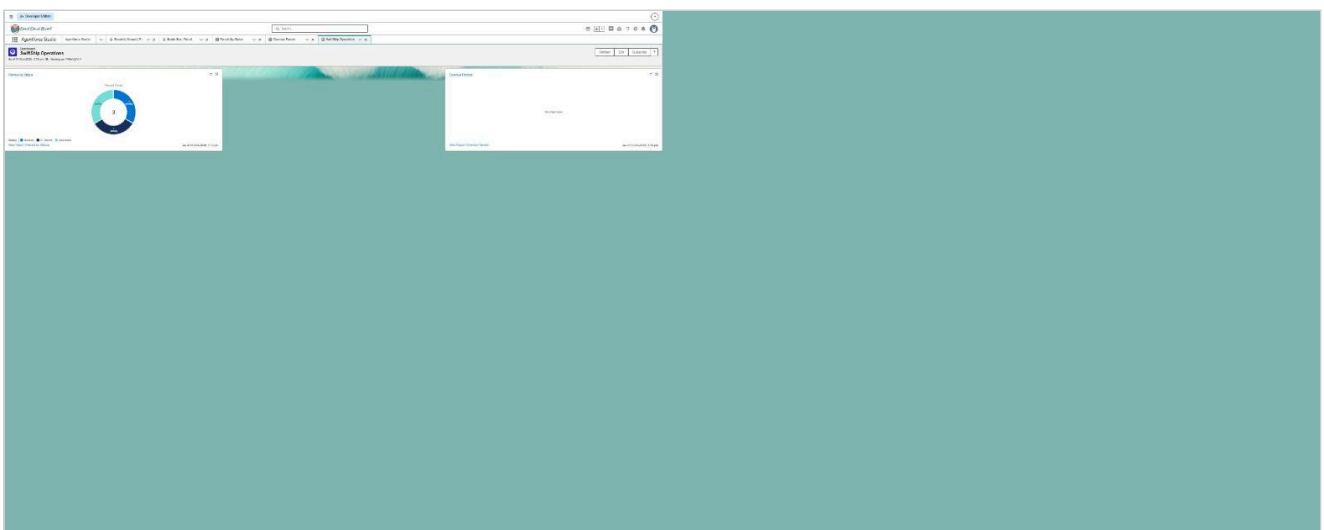


Figure 9: Agent preview: "Track parcel P-001"

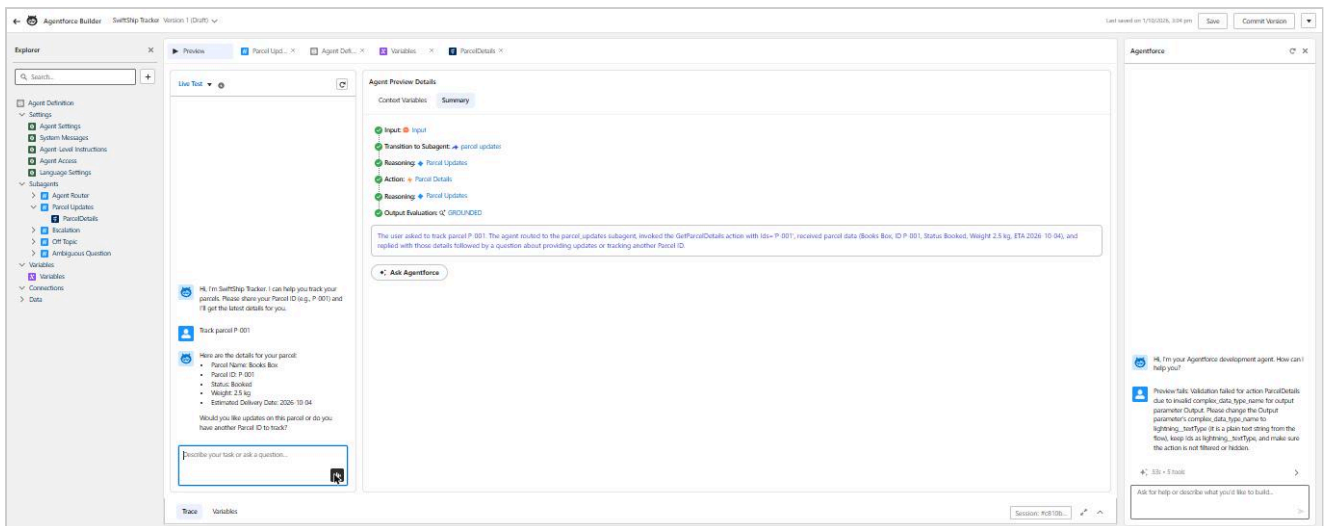


Figure 10: Agent SwiftShip Tracker, Version 1 (Active)

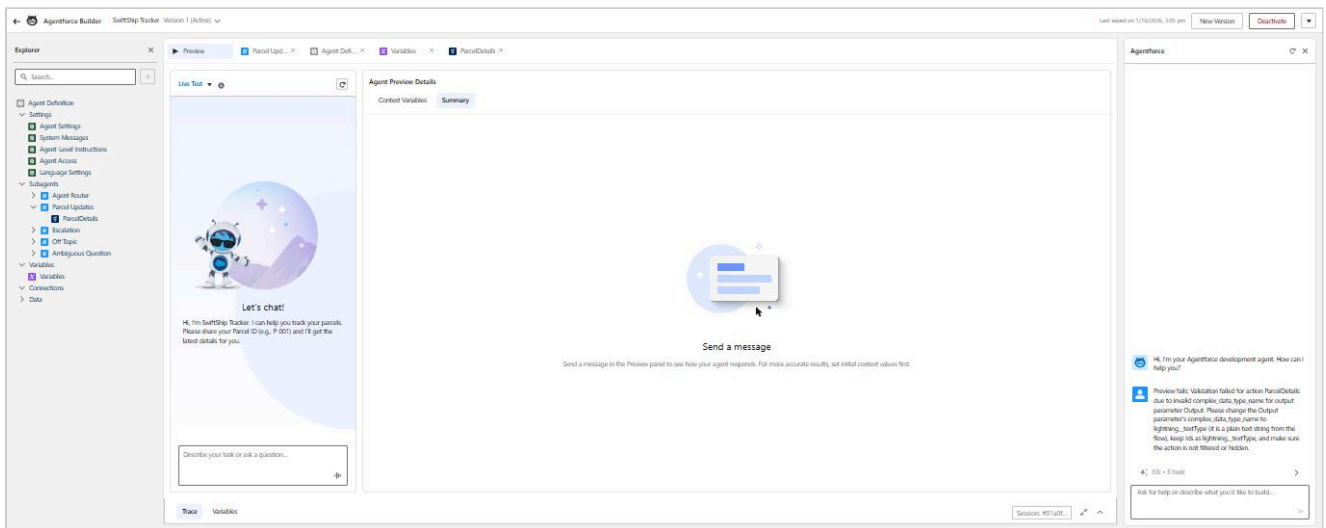


Figure 11: Prompt Builder preview: resolved prompt filled with P-001 data

